Rapport de TP — Séance 3 : Ingestion Réseau IoT sur Sockets TCP

Cours : Architecture & Ingestion Big Data | Date : 23 mai 2026
Implémentation d'un serveur distribué résilient et protocole NDJSON

1. Contexte & Objectifs du TP

Dans le prolongement des travaux sur l'intégrité des flux, cette séance formalise le passage à une architecture distribuée. Des stations météo et des capteurs d'irrigation agricoles distants transmettent désormais leurs métriques en temps réel à un service de collecte centralisé.

L'objectif de ce TP est de concevoir le serveur d'ingestion réseau capable de :

- **Établir des communications bas niveau** via des sockets TCP bruts en Python.
- **Standardiser les échanges** à l'aide d'un protocole orienté flux : le NDJSON (Newline Delimited JSON).
- **Valider une matrice de 10 capteurs industriels** transmettant des données nominales mélangées à des anomalies structurelles et logiques.
- **Mettre en place une couche d'observabilité de production** en capturant les métadonnées de traitement (Run Metadata) pour chaque transaction.

2. Architecture Modulaire du Projet

Le projet applique une séparation stricte des responsabilités au sein du répertoire seance3 :

- **data/sample_readings.json :** Jeu de données de 10 capteurs avec pannes simulées.
- **src/models.py :** Dataclasses d'échange et sérialisation/désérialisation sous forme de dictionnaires.
- **src/protocol.py :** Algorithme de découpage de trame (framing via recv_line) et protocole NDJSON.
- **src/validators.py :** Moteur d'évaluation des règles métier agricoles et cumul des erreurs.
- **src/server.py :** Démon d'écoute TCP, gestion des défaillances et journalisation des Run Metadata.
- **src/client.py :** Émetteur réseau et affichage du bilan d'ingestion structuré.
- **main_demo.py :** Suite de tests unitaires et assertions locales déconnectées du réseau.

3. Analyse du Protocole & Choix Techniques

3.1. Pourquoi le NDJSON plutôt qu'un JSON classique ?

Le choix du format NDJSON (Newline Delimited JSON) est crucial dans les architectures Big Data. Un fichier JSON standard structuré sous forme de tableau [...] impose à l'interpréteur de charger l'intégralité du document en mémoire vive avant de pouvoir démarrer le parsing.

Le NDJSON résout cette contrainte en séparant chaque enregistrement indépendant par un caractère de saut de ligne (\n). Le flux peut ainsi être lu et traité ligne par ligne (au fil de l'eau), offrant une empreinte mémoire constante, peu importe la taille du flux, et une excellente isolation des pannes.

3.2. Gestion du Framing Réseau (recv_line)

Les sockets TCP fonctionnent par flux continu d'octets (Byte Stream) sans notion innée de début ou de fin de message. Pour isoler nos messages, nous avons implémenté une fonction de découpage manuel basée sur un accumulateur de type bytearray. Le serveur lit le socket par blocs, cherche l'index du caractère b'\n', extrait la chaîne correspondante pour la traiter, et conserve le reliquat dans le tampon pour l'itération suivante.

4. Matrice de Validation des Données (10 Échantillons)

Le jeu de données simulé comportait 10 enregistrements complexes conçus pour valider les filtres métier :

#	Capteur (sensor_id)	Type	Statut	Cause de l'échec / Validation
1	temp_01	temperature	✅ **OK**	Température (23.5°C) nominale.
2	hum_01	humidity	✅ **OK**	Humidité (68.2%) dans les clous.
3	rain_01	rainfall	✅ **OK**	Pluviométrie (12.0 mm) cohérente.
4	temp_02	temperature	❌ **REJETÉ**	Valeur aberrante (-999.0°C) hors limites [-50, 60].
5	(vide)	humidity	❌ **REJETÉ**	Identifiant absent ou composé d'espaces.
6	hum_03	humidity	❌ **REJETÉ**	Type de données incohérent (str 'haute' au lieu de float).
7	irr_01	irrigation	❌ **REJETÉ**	Incohérence logique : pump_status=OFF mais débit > 0.
8	temp_03	temperature	❌ **REJETÉ**	Chaîne ('not-a-date') non conforme ISO 8601.
9	irr_02	irrigation	✅ **OK**	pump_status=ON et débit cohérent (3.1 mm).

10	wind_01	wind_speed	✓✓OK	Vitesse du vent (15.4 km/h) valide.

5. Journalisation des Logs & Observabilité Production

L'architecture intègre un identifiant unique request_id généré par UUID4, propagé du client vers le serveur pour assurer une traçabilité complète. Une ligne dédiée [RUN METADATA] enregistre les indicateurs de performance clés (durée en millisecondes, adresses IPs, volumes rejetés) à chaque transaction.

6. Conclusion

Cette séance a permis de valider avec succès le déploiement d'un micro-service réseau résilient. L'utilisation combinée des sockets TCP et du protocole de découpage NDJSON garantit une ingestion performante, capable de filtrer efficacement la donnée de mauvaise qualité (Bad Data) à la frontière du système sans impacter les messages sains, répondant ainsi aux exigences d'un pipeline Big Data moderne.

